Walden Turtle, React Version: Guide to the Code

This package turns the Walden map into a small React app. The map, the four lenses and the Overlay look and behave as before, but the code is split into short files, one for each part of the screen, instead of one 13 MB web page. That makes it much easier to read, change and reuse.

It works in two steps. Python walks the text and writes data files. A React app, which draws the map with the ArcGIS Maps SDK for JavaScript, reads those files and shows them.

What changed from the single-file version

Before (one HTML file)	Now (React app)
Everything in one `walden_lenses_arcgis.html`, with the data, picture, PDF and font pasted inside.	Data in separate files (`walden.json`, `basemap.jpg`, `walden.pdf`); code in about 15 short files.
Open it by double-clicking.	Run it with `npm run dev`, or publish the `dist` folder to a web host. Double-clicking does not work.
The SDK loaded from Esri's CDN in a script tag.	The SDK is installed from npm and packed into the app. A few support files still come from Esri's servers.
Controls built by hand-written HTML strings.	Controls written as React components, one per file.
Python assembled the web page.	Python writes data only; it never touches the page.

What is in the folder

File or folder	What it is	Do you edit it?
`build_data.py`	The control panel for step 1: start, place, text and basemap settings.	Yes, for settings
`START_HERE.txt`	The short run instructions.	No
`python/`	The turtle: `make_lenses.py` and the rule files for each lens.	Rarely
`data/`	The cleaned text, chapter list, PDF, 1906 map picture and the saved word and sentence analyses.	Only for another text
`vendor/fonts/`	The Cardo font.	No
`app/`	The React app. This is where you work on how the map looks and behaves.	Yes
`app/public/data/`	The files Python writes for the app to read.	No: rebuilt each time
`app/src/`	The app's code (see the tour below).	Yes

Quick start

You need two free programs: Python 3.10 or newer (python.org) and Node.js 20 or newer (nodejs.org). Node.js includes `npm`, which downloads the app's libraries.

Step 1: build the data (a few seconds).

1. In a terminal, go into the package folder: `cd walden_turtle_react`
2. Install the Python packages once: `pip install numpy pyproj pillow`
3. Run: `python3 build_data.py`

This writes `walden.json`, `basemap.jpg` and `walden.pdf` into `app/public/data/`, and the fonts into `app/public/fonts/`.

Step 2: run the app (about a minute the first time).

1. Go into the app folder: `cd app`

2. Download its libraries once: `npm install`
3. Start it: `npm run dev`
4. Open the address it prints in your browser, usually `http://localhost:5173`.

While `npm run dev` is running, any change you save to a file in `app/src/` appears in the browser within a second, with no rebuild. Press Ctrl+C in the terminal to stop it.

To publish it. Run `npm run build`. It makes the folder `app/dist`. Upload everything inside `dist` to a web host. To test the built version on your own computer, run `npm run preview`.

One thing to know: opening `dist/index.html` by double-clicking will show an error. Browsers refuse to load data files from a plain file on disk, so the app must be served by a web host or by `npm run dev` or `npm run preview`.

The map needs an internet connection, because the ArcGIS SDK fetches a few support files from Esri when it opens.

What you should see: the Punctuation lens with a blue-to-green path, the green Incipit marker at the hut, the red Finis marker at the end, and the Legend panel at the right. The 1906 basemap starts off; tick Basemap in the Legend to show it.

How the pieces fit

The whole app rests on one idea: **the screen is a function of the state**. The state is a single object, `s`, that holds every setting the controls can change. The controls and the map are both drawn from it.

Five React ideas

Idea	Plain meaning	Where you see it
Component	A function that returns a piece of the screen. Each is a file with a capitalized name.	`Legend.jsx`, `ModeBar.jsx`, `Footer.jsx`
JSX	HTML-looking text written inside JavaScript. `{ }` drops a value into it.	Every `.jsx` file

Idea	Plain meaning	Where you see it
Props	Information a component receives from its parent, like arguments to a function.	`<Legend D={D} s={s} set={set} />`
State	A value React remembers between redraws. Changing it redraws what depends on it.	`useState` in `Walden.jsx`
Effect	Code that runs after the screen draws or a value changes, to talk to something outside React.	`useEffect` in `Walden.jsx`, `Markers.jsx`, `Footer.jsx`

What happens when you click a button

Take clicking the Letter button.

1. `ModeBar.jsx` calls `set({ lens: "letter" })`.
2. `set` in `Walden.jsx` makes a new state object with `lens` changed and everything else kept.
3. React redraws every component that uses the state: the Letter button turns dark, the Legend shows the Letter rules, the plate reads "Plate II".
4. An effect in `Walden.jsx` notices `s.lens` changed and calls `map.apply(s)`. The map hides the Punctuation layers and shows the Letter layers.
5. A second effect calls `map.fit(s, true)`, and the view glides to fit the Letter path.

Why the map is a plain class, not a component

The map holds thousands of shapes. If React rebuilt them on every click the app would crawl. So `src/map/WaldenMap.js` builds every shape once, when the app starts, and afterward only switches them on and off and restyles them. React never sees a shape; it only calls three methods: `apply(s)`, `setBase(on, fade)` and `fit(s, animate)`. In the other direction, the map calls `onHover(...)` and React shows the popup.

This split also means you can change how the screen looks (React files) without touching how the map draws (the `map` folder), and the reverse.

A tour of the app's files

All of these are inside `app/`. Read them in this order: `Walden.jsx` first, then any component that interests you, then `map/WaldenMap.js`. Every file begins with a comment saying what it is for.

The screen (React)

File	What it does
`index.html`	The one web page. It holds an empty box, `<div id="root">`, that React fills.
`src/main.jsx`	Starts the app: loads the style sheets and puts `<App />` in the box.
`src/App.jsx`	Loads `data/walden.json`. Shows "Loading" until it arrives, then shows `<Walden />`.
`src/Walden.jsx`	The map screen. Holds the state `s`, creates the map, passes state down to the controls, and runs the effects that copy state into the map.
`src/components/ModeBar.jsx`	The top-left panel: One lens or Overlay, and the four lens buttons.
`src/components/ZoomBar.jsx`	The + and - buttons.
`src/components/Legend.jsx`	The right-hand panel: title, PDF link, Rules or Lenses, basemap and fade, Points, Turns, chapter list.
`src/components/Plates.jsx`	The bottom row: north arrow, plate number and caption, color ramp with its quotation tooltips.
`src/components/Footer.jsx`	The scale line, the basemap credit and "Powered by Esri".
`src/components/Markers.jsx`	The Incipit and Finis markers, kept pinned to their map positions.
`src/components/Popup.jsx`	The hover popup, placed above or below what you point at.

File	What it does
`src/styles.css`	The whole look: colors, panels, buttons, fonts, the paper effect.

The map (ArcGIS, no React)

File	What it does
`src/map/WaldenMap.js`	Creates the ArcGIS view, builds every path, point and chapter-start shape once, and offers `apply`, `setBase`, `fit`, `zoomBy`. Also handles hover and draws the turn arrows.
`src/map/symbols.js`	How things look: line color and width, point size and fill, the turn arrow shape.
`src/map/popupHtml.js`	The words in the popups, including the sentence text with its verbs in bold.
`src/map/geo.js`	Latitude and longitude to the SDK's map units and back, the zoom maths, small color helpers.

Configuration

File	What it does
`package.json`	Lists the libraries the app needs (React, the ArcGIS SDK, Vite) and the commands `dev`, `build` and `preview`.
`package-lock.json`	The exact versions that were tested. Leave it alone; `npm install` reads it.
`vite.config.js`	Settings for Vite, the tool that runs and packs the app.

The data block `D`

Everything Python knows about Walden reaches the app as one object, `D` (the contents of `walden.json`). `App.jsx` loads it and hands it down. Its main parts: `D.text` (the full text), `D.chapterTitles` and `D.chBounds` (chapter names and start positions), `D.lenses` (one entry per lens, with its chapters' paths, points, turn arrows, color and legend text), `D.ramp` (the 60-step chapter colors) and `D.basemap` (the picture's corner coordinates and file). You never edit it by hand: change the Python and run `build_data.py` again.

What to change, and where

Change one thing at a time. With `npm run dev` running, edits to files in `app/src/` show up in the browser at once. Edits to Python or to `build_data.py` need `python3 build_data.py` again, then a browser refresh.

Look and wording (React side)

To change	Where	Notes
Colors, fonts, panel borders, the paper effect	`src/styles.css`	The first lines define `--paper`, `--ink`, `--rubric` and others; changing one recolors everything that uses it.
Whether the basemap starts on, and its starting strength	`base: false, fade: 15` in `src/Walden.jsx`	`true` turns it on; fade is a percent.
The text on the plate	`src/components/Plates.jsx`	Search for "one meter to each".
The Legend's checkbox labels	`src/components/Legend.jsx`	For example "Points" and "Turns".
The footer credit	`src/components/Footer.jsx`, and `BASEMAP_CREDIT` in `build_data.py`	Keep "Powered by Esri".
How the screen is laid out (what is top left, bottom and so on)	The `return (...)` at the end of `src/Walden.jsx`	Each `className` such as `ctl tl` is a corner; its rules are in `styles.css`.

Map behavior (map side)

To change	Where	Notes
How the map fits on screen	`padL`, `padR`, `padT`, `padB` in `fit()` in `src/map/WaldenMap.js`	Pixels kept free on each side. `padR` leaves room for the Legend.
Zoom limits	`MIN_ZOOM`, `MAX_ZOOM` in `src/map/geo.js`	Currently 9 and 22.
Most turn arrows at once	`MAX_ARROWS` at the top of `WaldenMap.js`	Lower it if close zooms feel slow.
Point sizes	`radius` and `rsize` in `src/map/symbols.js`	`dotScale` and `Z100` set how points grow as you zoom in.
Chapter-start diamond size	`START_R` in `src/map/symbols.js`	Pixels at the base zoom.
Line thickness and opacity	`lineSym` in `src/map/symbols.js`	Width is `2 * PT`, in pixels.
Arrow shape and size	`arrowSym` in `src/map/symbols.js`	The two `[x, y]` lists are the shaft and the arrowhead.
Popup wording	`src/map/popupHtml.js`	
Popup width	the `w:` numbers in `describe()` in `WaldenMap.js`	Pixels.

Rules and colors (Python side)

To change	Where	Notes
Where the turtle starts, the title, the PDF, the basemap picture and its edges	`build_data.py`	Plain settings with comments.
Zoom at which turn arrows appear	`turnZoom` in the `LENSES` list of `python/make_lenses.py`	Punctuation 18, Letter 17.5, Word 20, Sentence 16. In the Overlay all four use the Punctuation value.

To change	Where	Notes
A lens's Overlay color	`color` in the same list	Also change its `quote`, which is checked against the book.
Shortest word that gets a point	`MIN_POINT` in `python/word_lens.py`	Currently 12 letters.
Shortest sentence that gets a point, and its size bands	`MIN_WORDS` and `BANDS` in `python/sentence_lens.py`	Currently 50 words.
Turn angles for words and sentences	`ANGLE` in `word_lens.py` and `sentence_lens.py`	Degrees clockwise from straight ahead.
Which marks turn the Punctuation turtle	`LOWER`, `UPPER`, `MIDDLE` in `python/walk_punct.py`	Lower marks turn right 90, upper marks turn left 90, middle marks turn right 45.
Chapter color ramp	`ANCHORS` in `python/ramp.py`	Run `python/check_ramp.py` afterward.

If you change how the Word or Sentence lens tags text, delete `data/word_tags.pkl` or `data/sent_tense.pkl` so they are made again. That needs spaCy (`pip install spacy`, then `python3 -m spacy download en_core_web_sm`) and takes a few minutes.

Using another text

The turtle works on any prose. Plan for a longer first build, because the Word and Sentence lenses must tag every word and sentence (a few minutes, plus the spaCy package).

1. **Copy the folder.** Make a second copy of the whole package for the new book, so the saved analyses in `data/` do not mix with Walden's.
2. **Prepare the text.** Put the prose in `data/clean.txt`: no page numbers, running headers, tables of contents or footnote markers. `python/clean_walden.py` shows how Walden was cleaned, but it is written for that one edition.
3. **Write the chapter list.** In `data/chapters.json`, list each chapter as `{"index": 1, "title": "...", "start": 0}`, where `start` is the character position in `clean.txt` where that chapter begins. A book with no chapters can use one entry.

4. **Delete the saved analyses.** Remove `data/word_tags.pkl` and `data/sent_tense.pkl`. They belong to Walden.
5. **Install the tagger.** `pip install spacy`, then `python3 -m spacy download en_core_web_sm`.
6. **Edit `build_data.py`.** Change `START_LAT`, `START_LON` and `START_NAME` to a place that fits the book, and `EPSG` to a metric grid for that region (epsg.io lists them; a UTM zone works anywhere). Change `TITLE`, `SOURCE_LABEL` and `SOURCE_PDF`. Set `SKIP_QUOTE_CHECK = True`.
7. **Basemap.** Either set `USE_BASEMAP = False`, or supply your own picture and its four edge coordinates. `python/prep_basemap.py` converts a georeferenced image into the right form; the picture must be north-up and in web mercator (EPSG 3857). With no basemap, the Basemap checkbox disappears by itself.
8. **Rewrite the quotes and captions.** Each Overlay lens color carries a quote about autumn from Walden (the `LENSES` list in `python/make_lenses.py`, and the ramp captions in `python/ramp.py`). Replace them with lines from your own book. Also change the colophon sentence "Set in Cardo. Drawn by a turtle that reads Walden more than one way." and the ramp caption "The colors of Walden Pond, in Thoreau's words" (in `src/components/Plates.jsx`).
9. **Build and look.** Run `python3 build_data.py`, then `npm run dev`. If the build stops with an error, the Troubleshooting section lists the common ones.

The path will drift. A long book can wander several kilometers from its start, so choose a start point with open space around it.

Publishing, versions, Esri's terms and testing

Publishing. `npm run build` makes `app/dist`. Upload its contents to any static web host: GitHub Pages, Netlify, a school web server. The build uses relative links, so it works from a folder as well as from a site's root.

Credits you must keep. The footer shows "Powered by Esri" through the SDK's attribution widget (`Footer.jsx`). Esri requires it, so do not remove it. The Gleason and Lincoln Town Archives basemap credit stays too.

No account or key. The map uses no Esri basemap, geocoding or other hosted service, so it needs no API key or sign-in. If you later add one, you will need a key and must follow Esri's terms for that service. Check Esri's current terms before using the site commercially.

The SDK version. `package.json` allows any 5.x version of `@arcgis/core`, but `package-lock.json` fixes the tested one (5.1.27), so `npm install` gives you the same code every time. One line must agree with it: `esriConfig.assetsPath` near the top of `src/map/WaldenMap.js` points to the `5.1` support files on Esri's servers. If you upgrade the SDK to a new minor version (say 5.2), change that number to match and look at all four lenses, the Overlay, hovering and the turn arrows before you publish. To avoid Esri's servers entirely, copy `node_modules/@arcgis/core/assets` into `app/public/assets` and set `assetsPath` to `"./assets"`.

A known warning. The footer uses the SDK's Attribution widget, which Esri has marked deprecated. It prints a notice in the browser console and works normally. If a future SDK version removes it, the app still runs but must show Esri's credit another way.

How it was tested. The sandbox this was built in could not reach Esri's servers, so the app was tested in a headless browser with Esri's support files supplied from a local copy of SDK 5.1.27. A fresh unzip, `npm ci` and `npm run build` completed. The Punctuation and Word lenses, the Overlay, hovering, turn arrows, the PDF link, the chapter "none" toggle and the basemap checkbox worked. The Letter and Sentence lenses use the same code but were not clicked through in this test. Open it once on your own computer, online, to confirm the live version behaves the same.

A handy tool. While the app runs, press F12 in the browser, open the Console tab and type `waldenMap.view.scale` or `waldenMap.view.goTo({center: [-71.34, 42.44], scale: 1000})`. The map object is exposed as `waldenMap` for experiments.

Troubleshooting

What you see	Likely cause	What to do
`python3: command not found`	Python is not installed, or has another name.	On Windows try `py build_data.py`. Otherwise install Python 3.10 or newer.
`ModuleNotFoundError: No module named 'numpy'` (or `pyproj`, `PIL`)	A Python add-on is missing.	Run `pip install numpy pyproj pillow`.

What you see	Likely cause	What to do
`npm: command not found`	Node.js is not installed.	Install Node.js 20 or newer from nodejs.org, then open a new terminal.
The browser shows "Could not load the data"	`app/public/data/walden.json` does not exist.	Run `python3 build_data.py` from the package folder, then refresh.
You double-clicked `dist/index.html` and see a blank page or an error	Browsers block data files on a plain file.	Use `npm run preview`, or upload `dist` to a web host.
The build stops with a line starting `AssertionError` mentioning a quote	The overlay quotes are not found in the text.	Set `SKIP_QUOTE_CHECK = True` for another book.
`FileNotFoundError` naming a file in `data/`	A path in `build_data.py` is wrong, or a file was moved.	Check the file exists where `build_data.py` says.
The Word or Sentence lens takes minutes, then errors about spaCy	The tagger is missing, or its saved results were deleted.	Install spaCy and `en_core_web_sm`, or restore the `.pkl` files in `data/`.
Paper-colored page, no path, errors mentioning `js.arcgis.com` in the console	The browser cannot reach Esri's servers, or a filter blocks them.	Check the connection and any school or work filter. Or host the support files yourself (see Publishing).
The path appears but no turn arrows	You are not zoomed in far enough.	Zoom in. Arrows appear at each lens's `turnZoom` (16 to 20); in the Overlay at 18.
Everything is slow when zoomed in	Too many arrows or points for the computer.	Lower `MAX_ARROWS` in `WaldenMap.js`.

What you see	Likely cause	What to do
The text looks like a default font, not Cardo	The font files are missing from `app/public/fonts/`.	Run `python3 build_data.py` again; it copies them.
A change in `src/` does not show	The dev server stopped, or the browser kept an old copy.	Check the terminal still says `npm run dev` is running, then press Ctrl+Shift+R (Cmd+Shift+R on a Mac).
A red error screen after you edit a `.jsx` file	A typo, often a missing `)` or `}` or an unclosed tag.	Read the first line of the message; it names the file and line. Undo your last edit to confirm that was the cause.

To see what the page is complaining about, press F12 in the browser and open the Console tab. A red line there usually names the exact problem. Paste it to Claude with the file name if you want help reading it.

Glossary

The map

Term	Meaning
Lens	One set of rules for how the turtle turns: Punctuation, Letter, Word or Sentence.
Turtle	The imaginary walker. It moves one meter per character and turns by the lens's rules.
Incipit, Finis	Latin for "it begins" and "the end": the start and end markers.
Point	A circle on the path where a run of marks, letters, or a long word or sentence occurs.

Term	Meaning
Chapter start	A diamond where each chapter's path begins.
Overlay	The view that draws all four lenses together, each in its own color.
Basemap	The 1906 Gleason map of Concord, placed under the paths by its corner coordinates.
EPSG	A number that names a map grid. 26986 is a metric grid for Massachusetts.
Web mercator	The projection the ArcGIS SDK draws in. Python works in meters on the EPSG grid; `geo.js` converts.
Graphic, GraphicsLayer	A single drawn shape, and a layer holding a group of them.
MediaLayer	The SDK layer type that places a picture on the map.
Symbol	The look of a shape: color, width, size. Defined in `symbols.js`.

The tools

Term	Meaning
React	A library for building a screen out of components that redraw when their state changes.
Component	A function that returns a piece of the screen. Files ending `.jsx`.
JSX	HTML-looking syntax written inside JavaScript.
Props	Values a component receives from its parent.
State	A value React remembers; changing it redraws what uses it.
Effect (`useEffect`)	Code that runs after a redraw, to connect React to something outside it, like the map.

Term	Meaning
Node.js	A program that runs JavaScript outside a browser; the tools below need it.
npm	Node's package manager. `npm install` downloads the libraries listed in `package.json`.
`node_modules`	The folder `npm install` fills with downloaded libraries. Never edit or share it; it is recreated at will.
Vite	The tool that serves the app while you work (`npm run dev`) and packs it for a host (`npm run build`).
`dist`	The folder `npm run build` makes: the finished site, ready to upload.
SDK	A library of ready-made code. Here, Esri's tools for drawing maps in a web page.
spaCy	A Python language tool that tags each word's type and helps find each sentence's tense.
Pickle (`.pkl`)	A Python file that stores results so they need not be computed again.